

PROJECT WORKFLOW & TEAM COLLABORATION PLATFORM

Complete Project Specification and Build Brief for AntiGravity

Version 1.0 | September 2026

Purpose: This document defines a complete web-based platform for managing projects through structured teams, sequential workflow stages, tasks, approvals, files, comments, notifications, deadlines, progress tracking, and final delivery. It is written as a build specification for AntiGravity.

1. Project Overview

The platform is a centralized project workflow management system. A client submits a project request, a Project Manager reviews and accepts it, creates a workflow, divides the work into stages, assigns teams and team members, and controls the movement of work from one team to the next.

The main concept is a controlled handoff workflow: one team completes its stage and submits its deliverables; the Project Manager reviews the submission; if approved, the next team is notified and can begin. If changes are required, the stage is returned to the responsible team with comments or notes.

2. Main Goals

- Centralize all project information in one platform.
- Create a clear workflow from project request to final delivery.
- Divide projects into stages and assign the correct team to each stage.
- Make responsibilities, deadlines, and deliverables clear.
- Prevent the next team from starting before the required handoff is approved.
- Provide transparent communication through comments, notes, and notifications.
- Keep project files organized with version history.
- Give managers real-time visibility into project progress and delays.
- Maintain an activity history for accountability and traceability.
- Provide a professional, scalable foundation for future features.

3. User Roles and Permissions

Role	Main Responsibility	Key Actions	Visibility
Client	Requests and follows projects	Create request, upload requirements, view status, comment, review final delivery	Own projects
Project Manager	Controls the entire project	Create workflow, assign teams, set deadlines, review stages, approve/reject, monitor progress	Full project

Team Leader	Manages one team stage	Assign tasks, monitor members, submit stage, upload deliverables, communicate	Assigned team/stage
Team Member	Completes assigned work	View tasks, update status, upload files, comment, respond to notes	Assigned work

4. Core Workflow

1. Client creates a Project Request.
2. Project Manager receives a notification and reviews the request.
3. Project Manager accepts the request and creates the project.
4. Project Manager defines project stages and their order.
5. Project Manager assigns a Team Leader and team members to each stage.
6. Project Manager sets deadlines and expected deliverables.
7. The first stage becomes ACTIVE.
8. Team Leader distributes tasks among team members.
9. Team members work, upload files, update task status, and communicate through comments.
10. Team Leader submits the completed stage.
11. Project Manager reviews the stage deliverables.
12. If approved, the stage becomes APPROVED and the next stage becomes ACTIVE.
13. If changes are required, the stage becomes REVISION REQUIRED and returns to the responsible team with feedback.
14. The process repeats until all stages are approved.
15. Project Manager marks the project READY FOR DELIVERY.
16. Client receives the final delivery and the project is marked COMPLETED.

5. Project Request Module

5.1 Client Request Form

The request form should contain:

- Project title
- Project description
- Business/problem statement
- Project type/category
- Priority
- Expected deadline
- Budget field (optional)
- Required features
- Expected deliverables
- Attachments
- Additional notes

5.2 Request Statuses

- Submitted
- Under Review
- Accepted
- Rejected
- Converted to Project

6. Project Management

Each accepted request becomes a project with:

- Unique project ID
- Project name and description
- Client
- Project Manager
- Overall status
- Priority
- Start date and target completion date
- Overall progress percentage
- Workflow stages
- Teams and members
- Files and versions
- Comments and notes
- Notifications
- Activity log

7. Workflow Builder

The Project Manager should be able to build a custom workflow rather than using one fixed sequence.

Example:

Planning → UI/UX Design → Development → Integration → Testing → Security Review → Deployment → Final Delivery

For every stage, allow the manager to define:

- Stage name
- Description
- Stage order
- Responsible team
- Team Leader
- Team members
- Start date
- Deadline
- Required deliverables
- Stage priority

- Approval requirement
- Dependencies

8. Team Management

Project Managers can create or select teams and assign members to each project stage.

- Team name
- Team Leader
- Team members
- Role of each member
- Current workload
- Assigned stage
- Team status

A member may belong to more than one team, but the system should clearly show active assignments and workload to reduce conflicts.

9. Task Management

- Create tasks within a stage.
- Assign each task to a team member.
- Set priority and deadline.
- Add description and acceptance criteria.
- Attach files.
- Add comments.
- Track status.
- Show overdue tasks.
- Allow the Team Leader to reassign tasks.

Task statuses: To Do → In Progress → Blocked → In Review → Completed

10. Stage Status and Approval

Status	Meaning
Locked	Stage cannot start yet because a previous dependency is incomplete.
Ready	Stage can start when activated.
Active	Team is currently working.
Submitted	Team has submitted the stage for review.
In Review	Project Manager is checking the deliverables.
Revision Required	Changes are required before approval.
Approved	Stage is accepted and completed.
Blocked	Work cannot continue because of an issue.

11. File and Version Management

Files must be attached to projects, stages, or tasks. Do not silently overwrite previous versions.

- Upload file
- File name and type
- Uploader
- Upload date/time
- Version number
- Related stage/task
- Optional description
- Download/view action
- Version history

12. Comments, Notes, and Communication

Comments

Comments support normal discussion between project participants and should be linked to the relevant project, stage, or task.

Notes

Notes are more formal instructions, feedback, or requirements. A note should show author, date/time, target stage/task, and whether it has been resolved.

Mentions

Allow users to mention another project participant using @name and trigger a notification.

13. Notification System

Create an in-app notification center. Notifications should be triggered by important project events.

- New project request
- Project accepted or rejected
- User assigned to a team
- New task assigned
- Task status changed
- Comment added
- User mentioned
- File uploaded
- Stage submitted
- Stage approved
- Stage returned for revision
- Next stage activated
- Deadline approaching
- Task or stage overdue
- Project completed
- Final delivery available

14. Dashboard Design

14.1 Project Manager Dashboard

- Total projects
- Active projects
- Completed projects
- Delayed projects
- Projects requiring review
- Upcoming deadlines
- Team workload
- Recent activity
- Notification center

14.2 Team Dashboard

- Active stage
- Team tasks
- Tasks by status
- Upcoming deadlines
- Overdue tasks
- Recent files
- Comments and notes
- Stage progress

14.3 Client Dashboard

- Submitted requests
- Active projects
- Project progress
- Current workflow stage
- Expected delivery date
- Messages/notes
- Final deliverables

15. Project Details Page

Use a clear tabbed or section-based layout:

- Overview
- Workflow
- Tasks
- Teams
- Files
- Comments
- Notes
- Activity
- Timeline

16. Timeline and Activity Log

The timeline should show major events chronologically. The activity log should record actions for accountability.

Example: Project created → Team assigned → Stage started → File uploaded → Stage submitted → Review completed → Stage approved → Next team notified.

17. Progress Calculation

Show progress at project, stage, and task levels. A simple implementation can calculate stage progress from completed tasks, while overall project progress can be calculated from approved workflow stages. The UI should clearly distinguish estimated progress from approved/completed progress.

18. Deadline and Delay Management

- Show deadlines on tasks and stages.
- Provide visual indicators for approaching deadlines.
- Mark overdue work automatically.
- Notify the Team Leader and Project Manager about overdue work.
- Show delay duration.
- Allow authorized users to update deadlines with an activity-log entry.

19. Search, Filter, and Sorting

Provide search and filters for projects, tasks, teams, users, statuses, priorities, and deadlines.

- Search by project name or ID
- Filter by status
- Filter by team
- Filter by Project Manager
- Filter by priority
- Filter by deadline
- Sort by newest, oldest, deadline, or progress

20. Access Control and Security

- Use role-based access control.
- Users should only see information allowed by their role and project assignment.
- Project Managers can manage their assigned projects.
- Team members should not approve their own stage submissions.
- Clients should not see internal team discussions unless explicitly marked as client-visible.
- Validate uploaded files and limit unsafe file types.
- Protect authentication and session data.
- Record important administrative actions in the activity log.

21. Suggested Database Structure

Use a relational data model or an equivalent structured backend. Suggested entities:

- Users
- Roles
- Projects
- ProjectRequests
- WorkflowStages
- Teams
- TeamMembers
- Tasks
- Files
- FileVersions
- Comments
- Notes
- Notifications
- ActivityLogs
- Approvals
- Deadlines

22. Important Relationships

- One Client can create many Project Requests.
- One Project Request can become one Project.
- One Project has many Workflow Stages.
- One Workflow Stage belongs to one responsible Team.
- One Team has many Team Members.
- One Stage has many Tasks.
- One Task can have multiple Files, Comments, and Notes.
- One Project has many Notifications and Activity Log records.
- One Stage can have multiple file versions and review actions.

23. Main Screens Required

#	Screen	Purpose
1	Login / Sign Up	Authentication and role-based entry.
2	Client Dashboard	Requests and project tracking.
3	Create Project Request	Submit project requirements.
4	Project Manager Dashboard	Portfolio overview and alerts.
5	Request Review	Accept, reject, or request clarification.
6	Create Project	Convert an accepted request into a project.
7	Workflow Builder	Create and order stages.
8	Team Assignment	Assign teams and members.
9	Project Overview	High-level project information.

10	Workflow Board	Visual stage-by-stage progress.
11	Stage Details	Tasks, files, comments, notes, and approval.
12	Task Details	Task information and updates.
13	Files / Versions	Deliverables and file history.
14	Notifications	All project alerts.
15	Activity Log	Chronological project history.
16	Final Delivery	Client-facing completed project.

24. UI/UX Requirements

- Use a modern, clean, professional SaaS-style interface.
- Use a consistent design system across all screens.
- Use clear status badges for workflow and task states.
- Use progress bars for projects and stages.
- Use cards for dashboard summaries.
- Use a sidebar for primary navigation.
- Make important actions visually clear.
- Use confirmation dialogs for destructive actions.
- Use responsive layouts for desktop, tablet, and mobile.
- Keep the interface simple enough for non-technical clients.

25. Recommended Visual Workflow

Display the workflow as connected stages/cards:

[Planning ✓] → [Design ✓] → [Development ●] → [Testing 🔒] → [Deployment 🔒] → [Delivery 🔒]

The active stage should be visually obvious. Locked stages should communicate that they are waiting for a dependency. Approved stages should remain accessible for history and review.

26. Example Scenario

A client requests a new website. The Project Manager accepts the request and creates the following workflow:

- Planning Team: requirements and project plan.
- Design Team: wireframes and visual design.
- Development Team: frontend and backend implementation.
- Testing Team: functional and usability testing.
- Deployment Team: production deployment.

When Planning submits its deliverables, the Project Manager reviews them. If approved, the Design Team receives a notification and its stage becomes active. If revisions are required, Planning receives the feedback and the next stage remains locked.

27. Edge Cases and Business Rules

- A stage cannot be approved without the required deliverables.

- A locked stage cannot be started manually unless an authorized Project Manager overrides the dependency.
- A Team Leader cannot approve their own stage.
- If a stage is returned for revision, the previous files remain available as historical versions.
- If a team member leaves a project, their completed work and activity history remain.
- If a deadline changes, record who changed it and why.
- If a project is cancelled, preserve its history and mark it CANCELLED rather than deleting it.
- If a required team member is unavailable, the Project Manager can reassign the task.
- Notifications should not be duplicated unnecessarily.
- All important state changes should create an activity-log entry.

28. Non-Functional Requirements

- Usability: The platform should be easy to understand with minimal training.
- Performance: Dashboard and project pages should load efficiently.
- Scalability: The architecture should support more projects, users, teams, and stages in the future.
- Security: Use authentication, authorization, validation, and secure data handling.
- Reliability: Project history and files should not be lost during normal operations.
- Maintainability: Use modular components and clear separation of concerns.
- Responsiveness: Support desktop, tablet, and mobile screen sizes.
- Accessibility: Use readable typography, sufficient contrast, clear labels, and keyboard-friendly controls where practical.

29. Functional Requirements Summary

- Users can authenticate according to their role.
- Clients can submit project requests.
- Project Managers can review and accept requests.
- Project Managers can create custom workflows.
- Project Managers can assign teams and members.
- Teams can manage and complete tasks.
- Users can upload files and maintain versions.
- Users can communicate through comments and notes.
- Project Managers can approve or return stages.
- The next team is notified automatically after approval.
- Users receive event-based notifications.
- The platform tracks progress, deadlines, and delays.
- The platform maintains an activity log.
- Clients can access final delivery.

30. Future Enhancements

- Email and push notifications.
- Calendar integration.
- Advanced analytics and project reports.

- Workload balancing across teams.
- Time tracking.
- Gantt chart view.
- Reusable workflow templates.
- Client approval portal.
- AI-assisted task assignment and deadline estimation.
- AI-generated project summaries.
- Automatic risk and delay detection.
- Integration with cloud storage and communication tools.
- Audit and compliance reporting.

31. AntiGravity Build Instructions

Build this as a complete working application, not as a static mockup. Implement the main user flows, navigation, role-based views, forms, state changes, workflow dependencies, file records, comments, notes, notifications, dashboards, approval logic, and activity history.

- Start by creating the data model and application architecture.
- Implement authentication and role-based access.
- Implement project request creation and review.
- Implement project creation and workflow builder.
- Implement teams, members, stages, and tasks.
- Implement stage dependencies and approval/revision logic.
- Implement files and version history.
- Implement comments, notes, mentions, and notifications.
- Implement dashboards and progress indicators.
- Implement deadlines, overdue states, and activity logs.
- Make the UI responsive and consistent.
- Use realistic sample data so every major screen can be demonstrated.
- Ensure empty states, loading states, validation messages, and error states are handled.
- Do not remove core functionality simply to simplify the demo.

32. Acceptance Criteria

- A client can submit a project request from start to finish.
- A Project Manager can convert an accepted request into a project.
- A Project Manager can create at least three workflow stages and assign different teams.
- The first stage can be activated while later stages remain locked.
- Team members can receive tasks and update them.
- A Team Leader can submit a stage for review.
- A Project Manager can approve or return a stage.
- Approval automatically activates the next stage and creates a notification.
- Revision creates feedback and keeps the next stage locked.
- Files and their versions are visible in the correct project/stage context.
- Comments and notes are visible to authorized participants.

- Deadlines and overdue states are displayed.
- The project progress updates as stages are completed.
- The activity log records major actions.
- The final stage can be completed and the project can be delivered to the client.
- The application remains usable on desktop and mobile layouts.

33. Final Product Vision

The final product should feel like a professional project operations platform where every project has a clear path, every team knows what to do, every handoff is controlled, and the Project Manager has complete visibility. The platform should reduce confusion caused by scattered messages and files by keeping tasks, teams, approvals, deliverables, deadlines, communication, and project history in one connected workflow.

34. Short Build Prompt

Build a complete responsive project workflow management web application based on this specification. The core flow is Client Request → Project Manager Review → Custom Workflow → Team Assignment → Tasks → Stage Submission → Manager Approval/Revision → Automatic Handoff Notification → Next Team → Final Delivery. Implement role-based access for Client, Project Manager, Team Leader, and Team Member. Include dashboards, workflow visualization, tasks, files with version history, comments, formal notes, notifications, deadlines, progress tracking, approvals, activity logs, search/filtering, responsive UI, validation, loading/error/empty states, and realistic demo data. Make the application functional and connected rather than a collection of static screens. Follow all functional, business, security, and non-functional requirements in this document.